

Lab Assignment-19.1

Name: CH VISHWANATH RAO

Hallticket:2303A51095

Batch-02

Task 1 – Python to Java Conversion

Task:

Translate a simple Python class into Java using AI assistance.

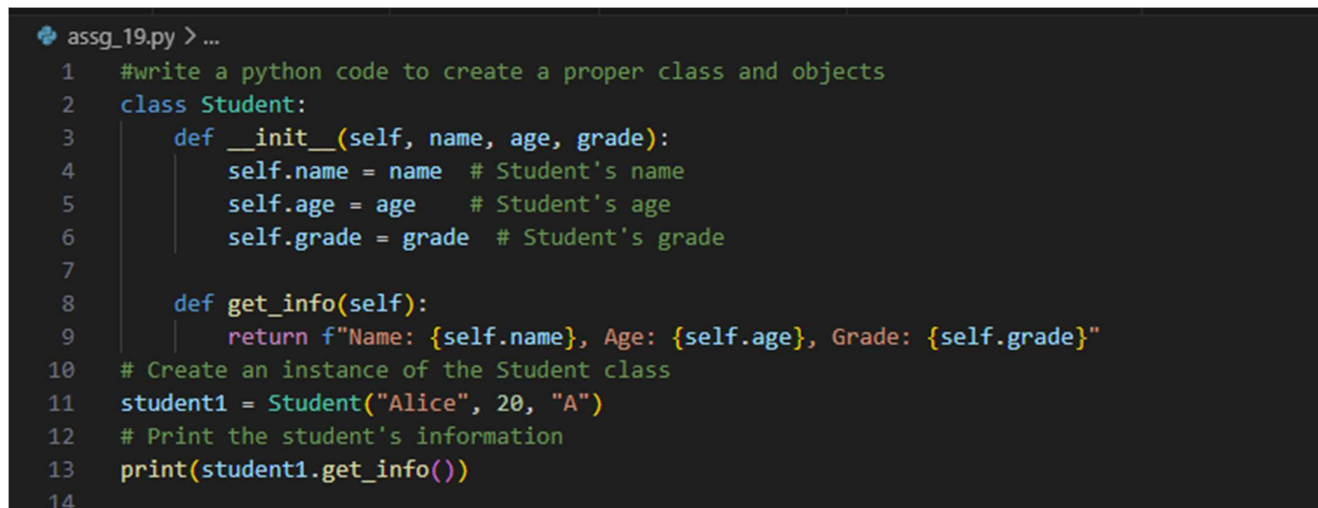
Instructions:

- Prompt AI to convert a Python class with attributes and methods into equivalent Java code.
- Verify that constructors, method signatures, and data types are properly handled.
- Run both versions (Python & Java) to compare expected outputs.

Expected Output:

- Equivalent working Java class translated from Python.

Screenshots:



```
assg_19.py > ...
1  #write a python code to create a proper class and objects
2  class Student:
3      def __init__(self, name, age, grade):
4          self.name = name # Student's name
5          self.age = age   # Student's age
6          self.grade = grade # Student's grade
7
8      def get_info(self):
9          return f"Name: {self.name}, Age: {self.age}, Grade: {self.grade}"
10 # Create an instance of the Student class
11 student1 = Student("Alice", 20, "A")
12 # Print the student's information
13 print(student1.get_info())
14
```

```

14
15 #convert the above into java code
16 public class Student {
17     private String name; // Student's name
18     private int age; // Student's age
19     private String grade; // Student's grade
20
21     // Constructor to initialize the student's attributes
22     public Student(String name, int age, String grade) {
23         this.name = name;
24         this.age = age;
25         this.grade = grade;
26     }
27
28     // Method to get the student's information
29     public String getInfo() {
30         return "Name: " + name + ", Age: " + age + ", Grade: " + grade;
31     }
32
33     // Main method to create an instance of Student and print info
34     public static void main(String[] args) {
35         Student student1 = new Student("Alice", 20, "A");
36         System.out.println(student1.getInfo());
37     }
38 }
39

```

Output:

```
Name: Alice, Age: 20, Grade: A
```

Task 2 – C++ to Python Function Conversion

Task:

Convert a C++ function for factorial calculation into Python.

Instructions:

- Ask AI to translate a recursive C++ factorial function into Python.
- Validate correctness by testing with sample inputs.
- Ensure Python version follows Pythonic conventions.

Expected Output:

- A correct Python implementation equivalent to the original C++ factorial function.

Screenshots:

```

41  #write a c++ code function that calculates the factorial of a number
42  #include <iostream>
43  using namespace std;
44  // Function to calculate factorial
45  unsigned long long factorial(int n) {
46      if (n < 0) {
47          throw invalid_argument("Factorial is not defined for negative numbers.");
48      }
49      unsigned long long result = 1;
50      for (int i = 1; i <= n; ++i) {
51          result *= i;
52      }
53      return result;
54  }
55  int main() {
56      int number;
57      cout << "Enter a number to calculate its factorial: ";
58      cin >> number;
59      try {
60          unsigned long long fact = factorial(number);
61          cout << "Factorial of " << number << " is " << fact << endl;
62      } catch (const invalid_argument& e) {
63          cout << "Error: " << e.what() << endl;
64      }
65      return 0;
66  }

```

```

67  #write above into python code
68  def factorial(n):
69      if n < 0:
70          raise ValueError("Factorial is not defined for negative numbers.")
71      result = 1
72      for i in range(1, n + 1):
73          result *= i
74      return result
75  try:
76      number = int(input("Enter a number to calculate its factorial: "))
77      fact = factorial(number)
78      print(f"Factorial of {number} is {fact}")
79  except ValueError as e:
80      print(f"Error: {e}")
81

```

Output:

```

Enter a number to calculate its factorial: 10
Factorial of 10 is 3628800

```

Task 3 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

Screenshots:

```
82 #write a sql commands that create a table named as employee and insert some data and retrieve the same and salary which are salary greater than 50000
83 -- Create the employee table
84 CREATE TABLE employee (
85     id INT PRIMARY KEY,
86     name VARCHAR(50),
87     position VARCHAR(50),
88     salary DECIMAL(10, 2)
89 );
90 -- Insert some data into the employee table
91 INSERT INTO employee (id, name, position, salary) VALUES
92 (1, 'John Doe', 'Software Engineer', 60000.00),
93 (2, 'Jane Smith', 'Data Analyst', 45000.00),
94 (3, 'Emily Davis', 'Project Manager', 75000.00),
95 (4, 'Michael Brown', 'Intern', 30000.00);
96
97 -- Retrieve employees with salary greater than 50000
98 SELECT * FROM employee WHERE salary > 50000;
99
```

```
100 #write the above sql commands into python using pandas dataframe
101 import pandas as pd
102 # Create a DataFrame to represent the employee table
103 data = {
104     'id': [1, 2, 3, 4],
105     'name': ['John Doe', 'Jane Smith', 'Emily Davis', 'Michael Brown'],
106     'position': ['Software Engineer', 'Data Analyst', 'Project Manager', 'Intern'],
107     'salary': [60000.00, 45000.00, 75000.00, 30000.00]
108 }
109 employee_df = pd.DataFrame(data)
110 # Retrieve employees with salary greater than 50000
111 high_salary_employees = employee_df[employee_df['salary'] > 50000]
112 print(high_salary_employees)
113
```

Output:

	id	name	position	salary
0	1	John Doe	Software Engineer	60000.0
2	3	Emily Davis	Project Manager	75000.0

Task 4 – Real-Time Application: Multi-Language Snippet

Converter

Scenario:

Students will choose a real-time use case where they need code in

multiple languages. Example:

- Converting a Python web scraping snippet into JavaScript for browser automation.
- Translating a Java method into C# for enterprise applications.

Instructions:

- Prompt AI to translate code between two chosen languages.
- Test both implementations to ensure they produce the same results.
- Document challenges faced during translation (e.g., syntax, libraries).

Expected Output:

- Working equivalent code snippets in at least two different programming languages.

SCREENSHOTS:

```
def fibonacci(n):
    if n <= 0:
        return []
    elif n == 1:
        return [0]
    elif n == 2:
        return [0, 1]

    first_unique = 0
    second_unique = 1
    fib_series = [first_unique, second_unique]

    for _ in range(2, n):
        next_unique = first_unique + second_unique
        fib_series.append(next_unique)
        first_unique = second_unique
        second_unique = next_unique

    return fib_series

try:
    n = int(input("Enter the number of terms: "))
    fib_series = fibonacci(n)
    print("Fibonacci series up to", n, "terms:")
    print(fib_series)
except ValueError:
    print("Please enter a valid integer.")
```

```

fibonacci.java
1  import java.util.ArrayList;
2  import java.util.List;
3  import java.util.Scanner;
4
5  public class Fibonacci {
6
7      public static List<Integer> fibonacci(int n) {
8          List<Integer> fibSeries = new ArrayList<>();
9
10         if (n <= 0) {
11             return fibSeries;
12         } else if (n == 1) {
13             fibSeries.add(0);
14             return fibSeries;
15         } else if (n == 2) {
16             fibSeries.add(0);
17             fibSeries.add(1);
18             return fibSeries;
19         }
20
21         int first = 0;
22         int second = 1;
23
24         fibSeries.add(first);
25         fibSeries.add(second);
26
27         for (int i = 2; i < n; i++) {
28             int next = first + second;
29             fibSeries.add(next);
30             first = second;
31             second = next;
32         }
33
34         return fibSeries;
35     }
36
37     public static void main(String[] args) {
38         Scanner scanner = new Scanner(System.in);
39
40         try {
41             System.out.print("Enter the number of terms: ");
42             int n = scanner.nextInt();
43
44             List<Integer> fibSeries = fibonacci(n);
45
46             System.out.println("Fibonacci series up to " + n + " terms:");
47             System.out.println(fibSeries);
48         } catch (Exception e) {
49             System.out.println("Please enter a valid integer.");
50         }
51
52         scanner.close();
53     }
54 }

```

Output:

```

PS C:\Users\chvis\OneDrive\Desktop\ai\assgg\lab_sample> cd "C:\Users\chvis\OneDrive\Desktop\ai\assgg\lab_sample";if ($?) { javac File.java } ; if ($?) { java File }

PS C:\Users\chvis\OneDrive\Desktop\ai\assgg\lab_sample> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/assgg/lab_sample/01.py
Enter the number of terms: 10
Fibonacci series up to 10 terms:
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg\lab_sample> 20
20
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg\lab_sample> & C:/Python314/python.exe c:/Users/chvis/OneDrive/Desktop/ai/assgg/lab_sample/01.py
Enter the number of terms: 20
Fibonacci series up to 20 terms:
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181]
PS C:\Users\chvis\OneDrive\Desktop\ai\assgg\lab_sample> =[]

```